



AWD COURSE

Pemrograman & Analisis Data

MODUL BAB 1

Dasar–Dasar Pemrograman Python

Panduan referensi komprehensif sintaksis dasar, tipe data, variabel, operator logika, percabangan kondisional, perulangan loop, pembuatan fungsi, dan standar penulisan kode bersih.

KATEGORI

Python & Data Science

FORMAT DOKUMEN

PDF

TINGKAT

Pemula s/d Menengah

AKSES ONLINE

course.awd.my.id

PEMATERI

I Putu Agus Wahyu Dupayana

AWD Course

Pembaruan: **September 2026**

Bab 1: Dasar Pemrograman Python & Algoritma

Awd Course

Modul Bab 1: Komentar, Tipe Data, Variabel, Operator, Percabangan, Perulangan, Fungsi, dan Latihan Kalkulator.

Komentar & Format Penulisan Kode Python

Dalam pemrograman Python, **komentar** adalah baris teks yang tidak akan dieksekusi oleh interpreter Python. Komentar berfungsi penting untuk:

1. Memberikan penjelasan dokumentasi logika kode kepada rekan pengembang.
2. Memudahkan pemeliharaan (*maintenance*) kode di masa mendatang.
3. Menonaktifkan sementara baris kode tertentu saat proses *debugging*.

1. Komentar Satu Baris (Single-Line Comment)

Komentar satu baris diawali dengan tanda pagar (`#`). Semua teks setelah tanda `#` hingga akhir baris akan diabaikan oleh Python.

PYTHON 3

Contoh Kode Praktik

```
# Ini adalah contoh komentar satu baris
print("Halo, selamat datang di kursus Python & Sains Data!") # Komentar di samping kode
```

OUTPUT EKSEKUSI:

```
Halo, selamat datang di kursus Python & Sains Data!
```

2. Komentar Multi Baris (Multi-Line Comment)

Untuk menuliskan penjelasan yang panjang atau berupa paragraf, kita dapat menggunakan beberapa tanda `#` berturut-turut atau menggunakan *triple quotes* (`'''` atau `"""`).

```
# Ini adalah komentar multi-baris
# yang menggunakan beberapa baris tanda pagar
# untuk menjelaskan alur algoritma yang kompleks.

"""
Ini adalah string multi-baris yang sering digunakan sebagai
docstring untuk mendokumentasikan fungsi, modul, atau kelas.
"""

print("Dokumentasi kode yang baik membuat program mudah dipahami.")
```

OUTPUT EKSEKUSI:

Dokumentasi kode yang baik membuat program mudah dipahami.

3. Best Practice Penulisan Komentar

- Tulis komentar yang menjelaskan **MENGAPA** kode tersebut ditulis, bukan hanya sekedar mengulang apa yang sudah jelas dari kode.
- Gunakan bahasa yang ringkas, jelas, dan mudah dipahami.
- Perbarui komentar jika ada perubahan logika pada kode.

```
# Menghitung diskon 15% untuk anggota premium
total_belanja = 500000
persen_diskon = 0.15
total_akhir = total_belanja * (1 - persen_diskon)

print(f"Total bayar setelah diskon: Rp {total_akhir:,.0f}")
```

OUTPUT EKSEKUSI:

Total bayar setelah diskon: Rp 425,000

Tipe Data dan Variabel Python

Variabel adalah lokasi memori yang diberi nama untuk menyimpan data. Python bersifat *dynamically typed*, yang artinya Anda tidak perlu mendeklarasikan tipe data secara eksplisit saat membuat variabel.

1. Tipe Data Primitif Dasar

Python memiliki 4 tipe data primitif utama:

- **Integer** (`int`): Bilangan bulat positif atau negatif tanpa desimal.
- **Float** (`float`): Bilangan riil dengan pecahan desimal.
- **String** (`str`): Teks atau deretan karakter yang diapit tanda kutip.

- **Boolean** (`bool`): Nilai kebenaran logika (`True` atau `False`).

PYTHON 3

Contoh Kode Praktik

```
# Deklarasi berbagai tipe data dasar
umur = 25                # int
tinggi_badan = 175.5     # float
nama_lengkap = "Budi Santoso" # str
is_active = True         # bool

# Memeriksa tipe data dengan fungsi type()
print("Tipe umur:", type(umur))
print("Tipe tinggi_badan:", type(tinggi_badan))
print("Tipe nama_lengkap:", type(nama_lengkap))
print("Tipe is_active:", type(is_active))
```

OUTPUT EKSEKUSI:

```
Tipe umur: <class 'int'>
Tipe tinggi_badan: <class 'float'>
Tipe nama_lengkap: <class 'str'>
Tipe is_active: <class 'bool'>
```

2. Struktur Data Koleksi (Collections)

Python menyediakan struktur data koleksi bawaan yang sangat fleksibel:

1. **List**: Terurut (*ordered*), dapat diubah (*mutable*), mengizinkan duplikasi. Ditulis dengan `[ ]`.
2. **Tuple**: Terurut (*ordered*), tidak dapat diubah (*immutable*). Ditulis dengan `( )`.
3. **Dictionary**: Pasangan kunci-nilai (*key-value pairs*), kunci harus unik. Ditulis dengan `{ }`.
4. **Set**: Kumpulan elemen unik tanpa urutan (*unordered*). Ditulis dengan `{ }`.

```
# 1. List (Daftar)
hobi = ["Membaca", "Koding", "Berenang"]
hobi.append("Musik")
print("List Hobi:", hobi)

# 2. Tuple (Pasangan data koordinat/tetap)
koordinat = (-7.250445, 112.768845) # Latitude, Longitude Surabaya
print("Koordinat:", koordinat)

# 3. Dictionary (Struktur Data Objek/Profil)
profil_pengguna = {
    "id": 101,
    "nama": "Siti Rahma",
    "pekerjaan": "Data Analyst",
    "keahlian": ["Python", "SQL", "Tableau"]
}
print("Profil Pengguna:", profil_pengguna["nama"], "-", profil_pengguna["pekerjaan"])

# 4. Set (Himpunan Unik)
angka_unik = {1, 2, 2, 3, 4, 4, 5}
print("Set angka unik:", angka_unik)
```

OUTPUT EKSEKUSI:

```
List Hobi: ['Membaca', 'Koding', 'Berenang', 'Musik']
Koordinat: (-7.250445, 112.768845)
Profil Pengguna: Siti Rahma - Data Analyst
Set angka unik: {1, 2, 3, 4, 5}
```

3. Konversi Tipe Data (Type Casting)

Anda dapat mengubah satu tipe data ke tipe data lain menggunakan fungsi bawaan seperti `int()`, `float()`, dan `str()`.

```
# Konversi dari string ke numerik
string_angka = "150"
angka_integer = int(string_angka)
angka_float = float(string_angka)

print("Hasil int:", angka_integer + 50)
print("Hasil float:", angka_float)
```

OUTPUT EKSEKUSI:

```
Hasil int: 200
Hasil float: 150.0
```

Operator Aritmatika, Perbandingan, & Logika

Operator adalah simbol khusus dalam Python yang digunakan untuk melakukan operasi pada nilai (*operand*).

1. Operator Aritmatika

Digunakan untuk operasi perhitungan matematika standar:

- Penjumlahan (`+`), Pengurangan (`-`), Perkalian (`*`), Pembagian (`/`)
- Pembagian Bulat (`//`), Modulo / Sisa Bagi (`%`), Pemangkatan (`**`)

PYTHON 3

Contoh Kode Praktik

```
a = 15
b = 4

print("Penjumlahan (a + b)      =", a + b)
print("Pengurangan (a - b)     =", a - b)
print("Perkalian (a * b)        =", a * b)
print("Pembagian (a / b)        =", a / b)
print("Pembagian Bulat (a // b) =", a // b)
print("Sisa Bagi / Modulo (a % b)=", a % b)
print("Pemangkatan (a ** b)     =", a ** b)
```

OUTPUT EKSEKUSI:

```
Penjumlahan (a + b)      = 19
Pengurangan (a - b)     = 11
Perkalian (a * b)        = 60
Pembagian (a / b)        = 3.75
Pembagian Bulat (a // b) = 3
Sisa Bagi / Modulo (a % b)= 3
Pemangkatan (a ** b)     = 50625
```

2. Operator Perbandingan (Relasional)

Menghasilkan nilai Boolean (`True` atau `False`) untuk membandingkan dua nilai:

- Sama dengan (`=`), Tidak sama dengan (`!=`)
- Lebih besar (`>`), Lebih kecil (`<`), Lebih besar sama dengan (`>=`), Lebih kecil sama dengan (`<=`)

```
x = 20
y = 35

print("x == y :", x == y)
print("x != y :", x != y)
print("x < y  :", x < y)
print("x >= y :", x >= y)
```

OUTPUT EKSEKUSI:

```
x == y : False
x != y : True
x < y  : True
x >= y : False
```

3. Operator Logika

Digunakan untuk menggabungkan kondisi kondisional:

- **and** : Bernilai **True** jika **kedua kondisi** benar.
- **or** : Bernilai **True** jika **salah satu kondisi** benar.
- **not** : Membalikkan nilai logika (**True** menjadi **False**, dan sebaliknya).

```
nilai_ujian = 85
kehadiran = 90

lulus_syarat = (nilai_ujian >= 75) and (kehadiran >= 80)
print("Apakah mahasiswa lulus?", lulus_syarat)
```

OUTPUT EKSEKUSI:

```
Apakah mahasiswa lulus? True
```

Percabangan Kondisional (If, Elif, Else, Match–Case)

Percabangan memungkinkan program membuat keputusan dan mengeksekusi blok kode tertentu berdasarkan kondisi Boolean.

1. Pernyataan **if**, **elif**, dan **else**

- **if** : Blok kode pertama yang dievaluasi.
- **elif** : Kondisi alternatif berikutnya jika kondisi sebelumnya tidak terpenuhi.
- **else** : Blok kode penutup yang dieksekusi jika seluruh kondisi di atas tidak terpenuhi.

```
nilai = 82

if nilai >= 85:
    grade = "A (Sangat Baik)"
elif nilai >= 75:
    grade = "B (Baik)"
elif nilai >= 60:
    grade = "C (Cukup)"
else:
    grade = "D (Perlu Perbaikan)"

print(f"Nilai: {nilai} -> Predikat: {grade}")
```

OUTPUT EKSEKUSI:

Nilai: 82 -> Predikat: B (Baik)

2. Percabangan Bersarang (Nested If)

Anda dapat menempatkan struktur `if` di dalam blok `if` lainnya untuk mengevaluasi kondisi bertingkat.

```
member = True
total_belanja = 600000

if member:
    if total_belanja >= 500000:
        diskon = 0.20 # Diskon 20%
    else:
        diskon = 0.10 # Diskon 10%
else:
    diskon = 0.05 # Diskon 5%

total_bayar = total_belanja * (1 - diskon)
print(f"Diskon: {diskon*100}% | Total Bayar: Rp {total_bayar:,.0f}")
```

OUTPUT EKSEKUSI:

Diskon: 20.0% | Total Bayar: Rp 480,000

3. Fitur Modern: `match-case` (Python 3.10+)

Mirip dengan struktur *switch-case* pada bahasa pemrograman lain, `match-case` mempermudah pencocokan pola nilai secara elegan.


```
kode_status = 200

match kode_status:
    case 200:
        pesan = "200 OK - Permintaan berhasil diproses."
    case 404:
        pesan = "404 Not Found - Sumber data tidak ditemukan."
    case 500:
        pesan = "500 Internal Server Error - Terjadi kesalahan server."
    case _:
        pesan = "Kode status tidak dikenali."

print(pesan)
```

OUTPUT EKSEKUSI:

200 OK - Permintaan berhasil diproses.

Struktur Perulangan (For & While Loop)

Perulangan (*looping*) digunakan untuk mengeksekusi blok kode berulang kali selama kondisi terpenuhi atau melintasi elemen-elemen dalam sebuah koleksi (*iterasi*).

1. Perulangan `for` dengan Fungsi `range()`

Fungsi `range(start, stop, step)` menghasilkan deret angkaurut.

```
# Melakukan perulangan 5 kali (dari 1 hingga 5)
print("Deret angka kuadrat:")
for i in range(1, 6):
    print(f"{i} kuadrat = {i**2}")
```

OUTPUT EKSEKUSI:

Deret angka kuadrat:
1 kuadrat = 1
2 kuadrat = 4
3 kuadrat = 9
4 kuadrat = 16
5 kuadrat = 25

2. Iterasi pada Elemen Koleksi (List & Dictionary)

PYTHON 3

Contoh Kode Praktik

```
bahasa_pemrograman = ["Python", "SQL", "JavaScript", "R", "Go"]

print("Daftar Bahasa:")
for idx, bahasa in enumerate(bahasa_pemrograman, start=1):
    print(f"{idx}. {bahasa}")
```

OUTPUT EKSEKUSI:

```
Daftar Bahasa:
1. Python
2. SQL
3. JavaScript
4. R
5. Go
```

3. Perulangan `while`

Perulangan `while` akan terus berjalan selama kondisi logika bernilai `True`. Pastikan selalu memperbarui variabel penghitung (*counter*) agar tidak terjadi perulangan tanpa henti (*infinite loop*).

PYTHON 3

Contoh Kode Praktik

```
hitung_mundur = 5

print("Memulai hitung mundur:")
while hitung_mundur > 0:
    print(f"{hitung_mundur}...")
    hitung_mundur -= 1

print("Peluncuran Sukses!")
```

OUTPUT EKSEKUSI:

```
Memulai hitung mundur:
5...
4...
3...
2...
1...
Peluncuran Sukses!
```

4. Kontrol Perulangan: `break`, `continue`, dan `pass`

- `break` : Menghentikan perulangan secara paksa.
- `continue` : Melewati iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya.
- `pass` : Pernyataan kosong sebagai penampung (*placeholder*).

```
# Contoh continue: Hanya mencetak bilangan ganjil
print("Bilangan ganjil antara 1 s.d. 10:")
for num in range(1, 11):
    if num % 2 == 0:
        continue # Lewati bilangan genap
    print(num, end=" ")
print()
```

OUTPUT EKSEKUSI:

```
Bilangan ganjil antara 1 s.d. 10:
1 3 5 7 9
```

Pembuatan & Pemanggilan Fungsi (Function)

Fungsi adalah blok kode terorganisir dan dapat digunakan kembali (*reusable*) untuk melakukan satu tugas spesifik.

1. Fungsi Bawaan (Built-in Functions)

Python menyediakan banyak fungsi siap pakai seperti `len()`, `max()`, `min()`, `sum()`, `abs()`, dan `round()`.

```
angka_list = [14, 28, 5, 92, 47, 63]

print("Jumlah Elemen :", len(angka_list))
print("Nilai Maksimum:", max(angka_list))
print("Nilai Minimum :", min(angka_list))
print("Total Jumlah :", sum(angka_list))
print("Rata-rata      :", round(sum(angka_list) / len(angka_list), 2))
```

OUTPUT EKSEKUSI:

```
Jumlah Elemen : 6
Nilai Maksimum: 92
Nilai Minimum : 5
Total Jumlah  : 249
Rata-rata      : 41.5
```

2. Membuat Fungsi Kustom (User-Defined Functions)

Fungsi didefinisikan menggunakan kata kunci `def`, diikuti nama fungsi dan tanda kurung parameter.

```
def sapa_pengguna(nama, peran="Pelajar"):  
    """Fungsi untuk menampilkan salam sambutan."""  
    return f"Halo {nama}, selamat belajar sebagai {peran} di Awd Course!"  
  
# Memanggil fungsi  
pesan_1 = sapa_pengguna("Agus", "Data Engineer")  
pesan_2 = sapa_pengguna("Budi") # Menggunakan default parameter  
  
print(pesan_1)  
print(pesan_2)
```

OUTPUT EKSEKUSI:

```
Halo Agus, selamat belajar sebagai Data Engineer di Awd Course!  
Halo Budi, selamat belajar sebagai Pelajar di Awd Course!
```

3. Fungsi dengan Nilai Balik Multi (Multiple Return Values)

```
def hitung_statistik_sederhana(data):  
    """Menghitung total, rata-rata, dan jangkauan (range)."""  
    total = sum(data)  
    rata_rata = total / len(data)  
    jangkauan = max(data) - min(data)  
    return total, rata_rata, jangkauan  
  
# Mengambil beberapa nilai balik sekaligus  
data_nilai = [80, 85, 90, 75, 95]  
tot, avg, rng = hitung_statistik_sederhana(data_nilai)  
  
print(f"Total: {tot} | Rata-rata: {avg:.2f} | Jangkauan: {rng}")
```

OUTPUT EKSEKUSI:

```
Total: 425 | Rata-rata: 85.00 | Jangkauan: 20
```

Tugas Praktik: Program Kalkulator Interaktif

Deskripsi Tugas:

Terapkan seluruh konsep yang telah Anda pelajari pada Bab 1 (Variabel, Tipe Data, Operator, Percabangan, dan Fungsi) untuk membuat program kalkulator fungsional.

Spesifikasi Program yang Diharapkan:

1. Memiliki fungsi matematika: `tambah(a, b)`, `kurang(a, b)`, `kali(a, b)`, dan `bagi(a, b)`. 2. Menangani pembagian dengan angka nol (*zero division error handling*). 3. Mengembalikan hasil komputasi yang rapi.

Tolok Ukur Hasil (Expected Output):

Setelah Anda melengkapi fungsi di bawah ini dengan benar, output yang diharapkan adalah: ```text 48 + 12 = 60 48 - 12 = 36 48 * 12 = 576 48 / 12 = 4.0 48 / 0 = Error: Pembagian dengan nol tidak diizinkan!`

PYTHON 3

Contoh Kode Praktik

```
# Tugas: Lengkapi bagian rumpang (_____) pada fungsi kalkulator di bawah ini:
```

```
# 1. Fungsi Penjumlahan
```

```
def tambah(a, b):  
    return _____
```

```
# 2. Fungsi Pengurangan
```

```
def kurang(a, b):  
    return _____
```

```
# 3. Fungsi Perkalian
```

```
def kali(a, b):  
    return _____
```

```
# 4. Fungsi Pembagian (dengan penanganan khusus jika pembagi adalah nol)
```

```
def bagi(a, b):  
    if b == _____:  
        return "Error: Pembagian dengan nol tidak diizinkan!"  
    return _____
```

```
# Uji Coba Fungsi Kalkulator (Jalankan setelah melengkapi fungsi di atas)
```

```
angka1 = 48  
angka2 = 12
```

```
print(f"{angka1} + {angka2} = {tambah(angka1, angka2)}")  
print(f"{angka1} - {angka2} = {kurang(angka1, angka2)}")  
print(f"{angka1} * {angka2} = {kali(angka1, angka2)}")  
print(f"{angka1} / {angka2} = {bagi(angka1, angka2)}")  
print(f"{angka1} / 0 = {bagi(angka1, 0)}")
```